UT1 · KOTLIN FUNDAMENTALS

Lambda expressions

Functions as values

What you will learn
Understand that in Kotlin a function can be stored in a variable, like a number or a text
Turn a normal function into a **lambda expression** step by step
Read and write **function types** such as `(Int) -> Int` or `() -> Unit`
Use the implicit parameter `it`
Write **higher-order functions**: functions that receive other functions as parameters
Use the **trailing lambda** syntax, the one Jetpack Compose uses everywhere

You already know how to declare and call functions with `fun`. In this block we take one more step: in Kotlin, a function is also a **value**. It can be stored in a variable, passed to another function as a parameter, and executed later. A function written as a value is called a **lambda expression**, or simply a **lambda**.

This is not just a curiosity. Jetpack Compose is built on lambdas: every button, every list and every screen you write will contain them. Look at this Compose code that you will write in a few weeks:

> **Jetpack Compose (preview, do not run it yet)**

```
Button(onClick = { counter++ }) {
    Text("Clicked $counter times")
}
```

The two blocks between braces `{ }` are lambdas: the first one says **what to do** when the button is clicked, and the second one says **what to draw** inside the button. By the end of this block you will be able to read this code and understand exactly what is happening.

Contents

Contents ..... **¡Error! Marcador no definido.**
1. From function to lambda 3
1.1 Anatomy of a lambda 3
1.2 More shapes of lambdas 4
2. Function types 4
3. The implicit parameter it 5
4. Higher-order functions 6
4.1 Passing a function you already have 6
5. Trailing lambdas 6
6. Lambdas you already have available 7
7. Lambdas in Jetpack Compose 8
8. Common mistakes 9
9. Summary 9
10. Exercises 11
Exercise 1 · From function to lambda 11
Exercise 2 · Name the type 11
Exercise 3 · Calculator with lambdas 11
Exercise 4 · Text analyser 12
Exercise 5 · Your own repeat 12
Exercise 6 · Predict the output 12
Challenge · Discount rules 13

1. From function to lambda

Let us start from a function you could write right now:

```
fun double(number: Int): Int {
    return number * 2
}

fun main() {
    println(double(5))   // 10
}
```

We are going to write the same behaviour as a lambda, removing pieces one at a time.

1. Remove `fun` and the name. The lambda has no name of its own; instead, we will store it in a variable.
2. Move the parameter **inside** the braces and separate it from the body with an arrow `->`.
3. Remove `return`. In a lambda, the value of the **last expression** is the result automatically.
4. Remove the return type. Kotlin infers it from the body.

```
fun main() {
    val double = { number: Int -> number * 2 }

    println(double(5))   // 10
}
```

The variable `double` now holds a function. You call it exactly like a normal function: with its name and parentheses.

1.1 Anatomy of a lambda

Part	In the example	Meaning
Braces	`{ … }`	A lambda always starts and ends with braces
Parameters	`number: Int`	The inputs, separated by commas. They can be omitted if there are none
Arrow	`->`	Separates the parameters from the body
Body	`number * 2`	The code to run. Its **last expression** is the value returned

Java

```
fun double(number: Int): Int {
    return number * 2
}
```

Kotlin

```
val double = { number: Int ->
    number * 2
}
```

1.2 More shapes of lambdas

A lambda can have any number of parameters, including none. When it has no parameters, the arrow is not written:

```
fun main() {
    val sayHello = { println("Hello!") }                    // no parameters
    val greet = { name: String -> "Hello, $name" }          // one parameter
    val add = { a: Int, b: Int -> a + b }                   // two parameters

    sayHello()
    println(greet("Laura"))   // Hello, Laura
    println(add(3, 4))        // 7
}
```

The body can have several lines. Remember the rule: the **last line** is the value the lambda returns (just like the branches of an `if` expression).

```
fun main() {
    val finalPrice = { price: Double, discount: Double ->
        val discountAmount = price * discount / 100
        println("You save $discountAmount")
        price - discountAmount          // this is the result
    }

    println(finalPrice(80.0, 25.0))    // You save 20.0, then 60.0
}
```

2. Function types

Every value in Kotlin has a type: `5` is an `Int`, `"Hi"` is a `String`. A lambda also has a type, called a **function type**. It describes the types of the parameters and the type of the result:

```
(ParameterTypes) -> ReturnType
```

Lambda	Function type	Read it as
`{ n: Int -> n * 2 }`	`(Int) -> Int`	receives an Int, returns an Int
`{ a: Int, b: Int -> a + b }`	`(Int, Int) -> Int`	receives two Ints, returns an Int
`{ name: String -> "Hi $name" }`	`(String) -> String`	receives a String, returns a String
`{ n: Int -> n % 2 == 0 }`	`(Int) -> Boolean`	receives an Int, returns a Boolean
`{ println("Hello") }`	`() -> Unit`	receives nothing, returns nothing useful

> **COMING FROM JAVA · Unit**
> `Unit` is the Kotlin equivalent of Java's `void`: it means "this function does not return anything useful". You will see `() -> Unit` constantly in Compose: it is the type of every `onClick`.

You can write the function type explicitly when declaring the variable, just like `val age: Int = 19`. When you do, Kotlin already knows the parameter types, so you can **omit them inside the lambda**:

```
fun main() {
    val add: (Int, Int) -> Int = { a, b -> a + b }
    val isAdult: (Int) -> Boolean = { age -> age >= 18 }
    val showMenu: () -> Unit = { println("1. Play  2. Exit") }

    println(add(2, 3))       // 5
    println(isAdult(16))     // false
    showMenu()
}
```

> **TIP · Where the types go**
> The types must appear **somewhere**: either in the variable (`val add: (Int, Int) -> Int = { a, b -> a + b }`) or in the parameters of the lambda (`val add = { a: Int, b: Int -> a + b }`). If they appear in neither place, Kotlin cannot infer them and the code does not compile.

3. The implicit parameter it

Lambdas with **exactly one parameter** are so common that Kotlin offers a shortcut: you can skip the parameter and the arrow, and refer to the parameter with the special name `it`.

```
fun main() {
    val double: (Int) -> Int = { number -> number * 2 }
    val doubleShort: (Int) -> Int = { it * 2 }      // same lambda

    println(double(4))        // 8
    println(doubleShort(4))   // 8
}
```

`it` only works when Kotlin already knows the type of the parameter, which normally happens because the function type is written somewhere (in the variable, or, as you will see in the next section, in the function that receives the lambda).

> **TIP · it or a name?**
> Use `it` for short, obvious lambdas: `{ it * 2 }`, `{ it > 0 }`. When the lambda is longer, or when there are lambdas inside lambdas, give the parameter a descriptive name: `{ price -> … }`. Your code will be much easier to read.

4. Higher-order functions

Now we can do something that was impossible before: write a function that receives **another function** as a parameter. A function that receives or returns functions is called a **higher-order function**.

Imagine a calculator. Instead of writing one function for each operation, we write a single function that receives the two numbers **and the operation to apply**:

```
fun calculate(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

fun main() {
    val sum = calculate(10, 5, { x, y -> x + y })
    val product = calculate(10, 5, { x, y -> x * y })
    val maximum = calculate(10, 5, { x, y -> if (x > y) x else y })

    println("$sum $product $maximum")   // 15 50 10
}
```

Look at the parameter `operation: (Int, Int) -> Int`. Its type is a function type: `calculate` does not know **which** operation it will run, only that it will receive two `Int` and return an `Int`. Whoever calls `calculate` decides the behaviour by passing a lambda. And because the function type is declared in `calculate`, the lambdas do not need to repeat the parameter types.

4.1 Passing a function you already have

If you already have a normal function with the right type, you can pass it with the **function reference** operator `::`:

```
fun subtract(a: Int, b: Int): Int = a - b

fun calculate(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

fun main() {
    println(calculate(10, 5, ::subtract))   // 5
}
```

Note: `::subtract` passes the function **without calling it**. Writing `subtract()` would try to run it immediately.

5. Trailing lambdas

This is the most important rule of the block, because Compose uses it on every line:

> **TIP · The trailing lambda rule**
> If the **last parameter** of a function is a function, the lambda can be written **outside the parentheses**.
> If that lambda is the **only** argument, the parentheses can be removed completely.

Let us apply it to our calculator. The three calls below are exactly the same:

```
fun calculate(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

fun main() {
    val r1 = calculate(10, 5, { x, y -> x + y })   // inside the parentheses
    val r2 = calculate(10, 5) { x, y -> x + y }    // trailing (recommended)

    val r3 = calculate(10, 5) { x, y ->            // trailing, several lines
        println("Adding $x and $y")
        x + y
    }
    println("$r1 $r2 $r3")   // 15 15 15
}
```

Kotlin already includes many functions designed to be used this way. For example, `repeat` runs a lambda a number of times. Its declaration is `repeat(times: Int, action: (Int) -> Unit)`: the last parameter is a function, so we write it as a trailing lambda. The lambda receives the number of the current repetition (starting at 0) as `it`:

```
fun main() {
    repeat(3) {
        println("Attempt ${it + 1}")
    }
}
```

Output

```
Attempt 1
Attempt 2
Attempt 3
```

> **COMING FROM JAVA · This is why Kotlin code looks like that**
> When you see `something { … }`, it is almost always **a function call whose last argument is a lambda**. It is not a new kind of block like `if` or `for`; it is just a function receiving a piece of code.

6. Lambdas you already have available

Text (`String`) has several functions that receive a lambda. The lambda is applied to **each character** of the text, and `it` is that character (`Char`):

Function	What it does	Example	Result
`count { }`	Counts the characters that meet the condition	`"banana".count { it == 'a' }`	`3`
`filter { }`	Keeps only the characters that meet the condition	`"R2-D2".filter { it.isDigit() }`	`"22"`
`any { }`	Is there **at least one** that meets it?	`"abc1".any { it.isDigit() }`	`true`
`all { }`	Do **all** of them meet it?	`"2026".all { it.isDigit() }`	`true`

The lambdas used here have the type `(Char) -> Boolean`: they receive a character and answer yes or no. A lambda that answers a yes/no question is called a **predicate**. With them we can, for example, validate a password, a very typical task in the login screen of an app:

```
const val MIN_LENGTH = 8

fun main() {
    val password = "Kotlin2026"

    val hasDigit = password.any { it.isDigit() }
    val hasUppercase = password.any { it.isUpperCase() }
    val longEnough = password.length >= MIN_LENGTH

    val isValid = hasDigit && hasUppercase && longEnough
    println("Valid password: $isValid")   // true
}
```

> **TIP · Preview: lists**
> In the collections block you will see that lists have the same functions, plus some more such as `map`. For example, `listOf(3, 8, 12).filter { it > 5 }` returns a list with `8` and `12`. Everything you learn about lambdas here applies directly there.

7. Lambdas in Jetpack Compose

Let us go back to the code from the beginning. You now have all the pieces to understand it:

Jetpack Compose (preview)
```
Button(onClick = { counter++ }) {
    Text("Clicked $counter times")
}
```

- `Button` is a **function** (Compose calls them composable functions).
- `onClick` is a parameter whose type is `() -> Unit`. We pass it a lambda with **named arguments**: `onClick = { counter++ }`. The lambda is not run now; the button keeps it and runs it every time the user clicks.

- The last parameter of `Button` is also a function: the content of the button. That is why it is written as a **trailing lambda** after the parentheses: `{ Text(…) }`.

The same happens with layouts. `Column` places its content one element under another, and that content is a trailing lambda. Since it is the only argument we pass, the parentheses disappear:

```
Jetpack Compose (preview)
Column {
    Text("Name: Laura")
    Text("Course: 2º DAM")
}
```

8. Common mistakes

Mistake	What happens	Fix
`{ n: Int -> return n * 2 }`	Does not compile: `return` is not allowed in a lambda like this	Remove `return`: the last expression is the result
`println(greet)` instead of `println(greet("Ana"))`	Prints a description of the function, not its result	Call it with parentheses and its arguments
`val double = { it * 2 }`	Does not compile: without a function type, Kotlin does not know there is a parameter, so `it` does not exist	Write the type: `val double: (Int) -> Int = { it * 2 }`
`{ a, b -> it + 1 }`	`it` only exists when there is **one** parameter and it has no name	Use the parameter names `a` and `b`
`calculate(10, 5, subtract)`	Does not compile: the function name alone is not a value	Use a reference: `::subtract`

9. Summary

Concept	Syntax
Lambda with parameters	`{ a: Int, b: Int -> a + b }`
Lambda without parameters	`{ println("Hi") }`
Function type	`(Int, Int) -> Int`, `(String) -> Boolean`, `() -> Unit`
Variable with function type	`val add: (Int, Int) -> Int = { a, b -> a + b }`
Implicit parameter	`val double: (Int) -> Int = { it * 2 }`
Higher-order function	`fun calculate(a: Int, b: Int, op: (Int, Int) -> Int)`

Concept	Syntax
Function reference	`calculate(10, 5, ::subtract)`
Trailing lambda	`calculate(10, 5) { x, y -> x + y }`
Only argument is a lambda	`repeat(3) { … }`, `Column { … }`

COMING FROM JAVA · Lambdas in Java

If you used lambdas in Java, the idea is the same, but the syntax changes: Java writes `(x) -> x * 2` and needs an interface such as `Function<Integer, Integer>` to store it. Kotlin puts the whole lambda between braces, `{ x -> x * 2 }`, and has real function types such as `(Int) -> Int`.

10. Exercises

Create one Kotlin file per exercise inside `UT1_KotlinBasics/src/exercises`. Write the function types explicitly the first time you use them: it is the best way to understand them.

Exercise 1 · From function to lambda

Rewrite each function as a lambda stored in a `val`. Then call both versions with the same arguments and check that they give the same result.

```
fun square(n: Int): Int {
    return n * n
}

fun isEven(n: Int): Boolean {
    return n % 2 == 0
}

fun fullName(name: String, surname: String): String {
    return "$name $surname"
}

fun printSeparator() {
    println("--------------------")
}
```

Finally, write the version of `square` and `isEven` that uses `it`.

Exercise 2 · Name the type

Write the function type of each lambda. Then check your answer in IntelliJ by declaring the variable with that type.

```
val a = { x: Double -> x / 2 }
val b = { text: String -> text.length }
val c = { name: String, age: Int -> "$name is $age" }
val d = { println("Loading...") }
val e = { n: Int -> n in 1..10 }
```

Exercise 3 · Calculator with lambdas

Write the higher-order function `calculate(a: Double, b: Double, operation: (Double, Double) -> Double): Double`.

- Declare four lambdas, `add`, `subtract`, `multiply` and `divide`, each with an explicit function type.
- Ask the user for two numbers and an operator (`+`, `-`, `*`, `/`). Use a `when` **expression** to choose the right lambda and store it in a variable, then call `calculate` with it.
- If the operator is not valid, show an error message.

Example

```
First number: 12
Second number: 4
Operator (+ - * /): *
12.0 * 4.0 = 48.0
```

TIP · Think about it

What is the type of the variable that stores the result of the `when`? Write it explicitly.

Exercise 4 · Text analyser

Ask the user for a sentence and display, using `count`, `filter`, `any` and `all` with lambdas:

- How many vowels it contains (hint: `it.lowercaseChar() in "aeiou"`).
- How many spaces it contains.
- The sentence with only its letters (hint: `it.isLetter()`).
- Whether it contains any digit.
- Whether all its letters are uppercase (hint: first keep only the letters).

Exercise 5 · Your own repeat

Write the function `repeatAction(times: Int, action: (Int) -> Unit)`, which runs `action` the number of times indicated, passing it the number of the repetition **starting at 1**. Use a `for` loop inside. Then call it with a trailing lambda to print:

Expected output

```
Lap 1 of 3
Lap 2 of 3
Lap 3 of 3
```

Exercise 6 · Predict the output

Without running the code, write down what it prints. Then run it and check your answers.

```
fun transform(text: String, rule: (String) -> String): String = rule(text)

fun main() {
    val shout: (String) -> String = { it.uppercase() + "!" }
    val whisper: (String) -> String = { "..." + it.lowercase() }

    println(transform("Hello", shout))
    println(transform("Hello", whisper))
    println(transform("Hello") { it.reversed() })
    println(transform("Hello") { text -> text.length.toString() })

    var total = 0
    repeat(4) { total += it }
    println(total)
}
```

Challenge · Discount rules

A shop applies different discount rules. Each rule is a lambda of type `(Double) -> Double` that receives a price and returns the new price.

- Write the function `applyRule(price: Double, rule: (Double) -> Double): Double`.
- Declare at least three rules: `noDiscount`, `blackFriday` (30 % off) and `studentDiscount` (10 % off, but only for prices of 20 € or more). Store the percentages in constants.
- Show the final price of a 50 € product and of a 15 € product with each rule, formatted with two decimals.
- Extra: write `combineRules(first: (Double) -> Double, second: (Double) -> Double): (Double) -> Double`, a function that **returns** a new lambda which applies both rules one after the other. Use it to apply Black Friday and the student discount together.